DSBDAL PRACTICAL FILE
Programs 1 to 10

Complete Python Codes with Line-by-Line Explanation

Designed for Jupyter Notebook execution using local device CSV files

Subject	Data Science and Big Data Analytics Laboratory (DSBDAL)
Platform	Python, Jupyter Notebook, pandas, NumPy, scikit-learn, seaborn, matplotlib, NLTK
Dataset Folder	C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS
Prepared Date	10 May 2026

How to Use This Document

- Run the programs cell-by-cell in Jupyter Notebook for easy execution and oral explanation.
- Every dataset is loaded from the local device folder. If your file name is different, change only the CSV file name in pd.read_csv().
- Program 3 uses Employee_Salary_Dataset.csv, whose actual columns are ID, Experience_Years, Age, Gender, and Salary.
- For Titanic1.csv, use the column class for passenger class. Do not use pclass for this file.
- The explanations are written in practical/viva-friendly language.

Program Index

This index shows all practical programs included in the document and the dataset used in each program.

No.	Practical / Program	Main Topic	Dataset / File
1	**Data Wrangling I**	Perform basic data wrangling, check statistics, format variables, normalize numeric data, and convert categorical variables into quantitative variables.	AcademicPerformanceNEW.csv from local device
2	**Data Wrangling II**	Clean missing values and inconsistencies, detect and handle outliers, and apply data transformation on an academic performance dataset.	AcademicPerformanceNEW.csv from local device
3	**Descriptive Statistics - Measures of Central Tendency and Variability**	Calculate mean, median, minimum, maximum, and standard deviation grouped by a categorical variable, and display statistical details of Iris species.	Employee_Salary_Dataset.csv and iris.csv from local device
4	**Data Analytics I - Linear Regression**	Build a Linear Regression model to predict house prices using the Boston Housing dataset.	BostonHousing.csv from local device
5	**Data Analytics II - Logistic Regression**	Implement logistic regression on Social_Network_Ads.csv and compute confusion matrix, TP, FP, TN, FN, accuracy, error rate, precision, and recall.	Social_Network_Ads.csv from local device
6	**Data Analytics III - Naive Bayes Classification**	Implement Naive Bayes classification on iris.csv and compute confusion matrix and classification measures.	iris.csv from local device
7	**Text Analytics**	Apply tokenization, POS	Sample text document inside

		tagging, stop words removal, stemming, lemmatization, and calculate TF, IDF, and TF-IDF.	Python
8	**Data Visualization I - Titanic Dataset**	Use Seaborn to find patterns in Titanic data and plot fare distribution histogram.	Titanic1.csv from local device
9	**Data Visualization II - Titanic Boxplot**	Plot a boxplot for distribution of age with respect to gender and survival status.	Titanic1.csv from local device
10	**Data Visualization III - Iris Feature Distributions**	List feature types, plot histograms and boxplots for Iris features, compare distributions, and identify outliers.	iris.csv from local device

Dataset Mapping

Dataset / File	Used In
AcademicPerformanceNEW.csv	Data Wrangling I and II
Employee_Salary_Dataset.csv	Descriptive Statistics - Part 1
iris.csv	Descriptive Statistics - Part 2, Naive Bayes, Data Visualization III
BostonHousing.csv	Linear Regression
Social_Network_Ads.csv	Logistic Regression
Titanic1.csv	Data Visualization I and II
Sample document inside Python	Text Analytics

PROGRAM 1	Data Wrangling I
Aim	Perform basic data wrangling, check statistics, format variables, normalize numeric data, and convert categorical variables into quantitative variables.
Dataset / File Used	AcademicPerformanceNEW.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\AcademicPerformanceNEW.csv")
df.columns = df.columns.str.strip().str.lower()

print("Dataset loaded successfully")
print(df.head())
print(df.info())
print(df.shape)
print(df.isnull().sum())
print(df.describe())
print(df.describe(include="all"))

variable_description = pd.DataFrame({
    "Variable": df.columns,
    "Data Type": df.dtypes.values,
    "Missing Values": df.isnull().sum().values,
    "Unique Values": df.nunique().values
})
print(variable_description)

numeric_columns = df.select_dtypes(include=[np.number]).columns
categorical_columns = df.select_dtypes(include=["object"]).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())

for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

for col in categorical_columns:
    df[col] = df[col].astype("category")

scaler = MinMaxScaler()
df_normalized = df.copy()
df_normalized[numeric_columns] = scaler.fit_transform(df[numeric_columns])
print(df_normalized.head())
```

```
df_encoded = df.copy()
label_encoder = LabelEncoder()
for col in categorical_columns:
    df_encoded[col] = label_encoder.fit_transform(df_encoded[col].astype(str))
print(df_encoded.head())

df_one_hot = pd.get_dummies(df, columns=categorical_columns)
print(df_one_hot.head())
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	import pandas as pd	Imports pandas to load CSV files and work with DataFrames.
2	import numpy as np	Imports NumPy for numeric operations and selecting numeric columns.
3	from sklearn.preprocessing import LabelEncoder	Imports LabelEncoder to convert categorical values into numbers.
4	from sklearn.preprocessing import MinMaxScaler	Imports MinMaxScaler to normalize numeric columns between 0 and 1.
5	BASE_PATH = ...	Stores the folder path where your CSV files are saved on your device.
6	pd.read_csv(...)	Loads AcademicPerformanceNEW.csv into a pandas DataFrame.
7	df.columns = ...	Removes extra spaces and converts column names to lowercase.
8	df.head()	Displays the first five rows of the dataset.
9	df.info()	Shows column names, data types, and non-null values.
10	df.shape	Shows number of rows and columns.
11	df.isnull().sum()	Counts missing values in each column.
12	df.describe()	Displays summary statistics for numeric columns.
13	df.describe(include="all")	Displays summary statistics for both numeric and categorical columns.
14	variable_description = pd.DataFrame(...)	Creates a table containing variable names, data types, missing values, and unique values.
15	select_dtypes(include=[np.number])	Selects numeric columns from the dataset.

16	select_dtypes(include=["object"])	Selects categorical/text columns from the dataset.
17	fillna(df[col].median())	Fills missing numeric values using the median.
18	fillna(df[col].mode()[0])	Fills missing categorical values using the most repeated value.
19	astype("category")	Converts text columns into categorical data type.
20	scaler = MinMaxScaler()	Creates a normalizer object.
21	fit_transform(...)	Normalizes numeric columns to a 0 to 1 range.
22	df_encoded = df.copy()	Creates a copy of the cleaned dataset for label encoding.
23	LabelEncoder()	Creates label encoder object.
24	fit_transform(...)	Converts each categorical column into numeric codes.
25	pd.get_dummies(...)	Creates one-hot encoded columns for categorical variables.

Conclusion

Data Wrangling I is completed by loading local CSV data, checking missing values and statistics, formatting variables, normalizing numeric values, and encoding categorical variables.

PROGRAM 2	Data Wrangling II
Aim	Clean missing values and inconsistencies, detect and handle outliers, and apply data transformation on an academic performance dataset.
Dataset / File Used	AcademicPerformanceNEW.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\AcademicPerformanceNEW.csv")
df.columns = df.columns.str.strip().str.lower()

print(df.head())
print(df.isnull().sum())

categorical_columns = df.select_dtypes(include=["object"]).columns
for col in categorical_columns:
    df[col] = df[col].astype(str).str.strip()

df = df.replace(["", "nan", "NaN", "None", "NONE", "?", "-", "NA", "N/A"], np.nan)

df["gender"] = df["gender"].replace({
    "M": "Male", "m": "Male", "male": "Male", "MALE": "Male",
    "F": "Female", "f": "Female", "female": "Female", "FEMALE": "Female"
})

score_columns = ["math score", "reading score", "writing score", "placement score"]
for col in score_columns:
    df.loc[(df[col] < 0) | (df[col] > 100), col] = np.nan

numeric_columns = df.select_dtypes(include=[np.number]).columns
categorical_columns = df.select_dtypes(include=["object"]).columns

for col in numeric_columns:
    df[col] = df[col].fillna(df[col].median())
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

outlier_columns = ["math score", "reading score", "writing score", "placement score", "placement count"]
plt.boxplot(df[outlier_columns])
plt.xticks(range(1, len(outlier_columns) + 1), outlier_columns, rotation=45)
plt.title("Boxplot Before Handling Outliers")
plt.show()

def detect_outliers_iqr(data, column):
```

```
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
    return data[(data[column] < lower_limit) | (data[column] > upper_limit)]

for col in outlier_columns:
    print("Outliers in", col)
    print(detect_outliers_iqr(df, col)[[col]])

def cap_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
    data[column] = np.where(data[column] < lower_limit, lower_limit, data[column])
    data[column] = np.where(data[column] > upper_limit, upper_limit, data[column])

for col in outlier_columns:
    cap_outliers_iqr(df, col)

skew_values = df[outlier_columns].skew()
transform_column = skew_values.abs().idxmax()

if df[transform_column].min() <= 0:
    shifted_data = df[transform_column] - df[transform_column].min() + 1
else:
    shifted_data = df[transform_column]

df[transform_column + "_Log"] = np.log1p(shifted_data)
print(df.head())
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	import matplotlib.pyplot as plt	Imports Matplotlib to draw boxplots and histograms.
2	pd.read_csv(...)	Loads the academic performance dataset from the local device.
3	df.columns = ...	Standardizes column names.
4	df.isnull().sum()	Scans every column for missing values.
5	categorical_columns = ...	Finds all text columns.
6	astype(str).str.strip()	Removes extra spaces from text values.
7	df.replace([...], np.nan)	Converts wrong missing symbols into proper NaN values.
8	df["gender"].replace(...)	Fixes inconsistent gender values such as M, male, F, and

		female.
9	score_columns = [...]	Stores columns that must contain marks or scores from 0 to 100.
10	df.loc[(df[col] < 0) | (df[col] > 100), col] = np.nan	Converts invalid score values into missing values.
11	fillna(median)	Fills numeric missing values using median.
12	fillna(mode)	Fills categorical missing values using mode.
13	outlier_columns = [...]	Selects numeric columns for outlier detection.
14	plt.boxplot(...)	Shows outliers visually before treatment.
15	quantile(0.25)	Calculates first quartile Q1.
16	quantile(0.75)	Calculates third quartile Q3.
17	IQR = Q3 - Q1	Calculates interquartile range.
18	lower_limit / upper_limit	Calculates outlier boundaries.
19	detect_outliers_iqr(...)	Finds rows containing outlier values.
20	np.where(...)	Caps outliers to lower or upper limit.
21	df[outlier_columns].skew()	Calculates skewness of numeric columns.
22	idxmax()	Selects the column with highest skewness.
23	np.log1p(...)	Applies log transformation to reduce skewness.

Conclusion

Data Wrangling II is completed by cleaning inconsistent values, filling missing values, detecting outliers using IQR, capping outliers, and applying log transformation.

PROGRAM 3	Descriptive Statistics - Measures of Central Tendency and Variability
Aim	Calculate mean, median, minimum, maximum, and standard deviation grouped by a categorical variable, and display statistical details of Iris species.
Dataset / File Used	Employee_Salary_Dataset.csv and iris.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"

df = pd.read_csv(BASE_PATH + r"\Employee_Salary_Dataset.csv")
df.columns = df.columns.str.strip().str.lower()

print(df.head())
print(df.info())
print(df.isnull().sum())

group_column = "gender"
numeric_variables = ["age", "experience_years", "salary"]

summary_statistics = df.groupby(group_column)[numeric_variables].agg([
    "mean", "median", "min", "max", "std"
])
print(summary_statistics)

salary_list_by_gender = df.groupby("gender")["salary"].apply(list)
print(salary_list_by_gender)

print("Mean:", df["salary"].mean())
print("Median:", df["salary"].median())
print("Minimum:", df["salary"].min())
print("Maximum:", df["salary"].max())
print("Standard Deviation:", df["salary"].std())

iris = pd.read_csv(BASE_PATH + r"\iris.csv")
iris.columns = iris.columns.str.strip().str.lower()

if "id" in iris.columns:
    iris = iris.drop("id", axis=1)

print(iris["species"].unique())
iris_numeric_columns = iris.select_dtypes(include=[np.number]).columns

iris_setosa = iris[iris["species"] == "Iris-setosa"]
iris_versicolor = iris[iris["species"] == "Iris-versicolor"]
```

```
iris_virginica = iris[iris["species"] == "Iris-virginica"]

print(iris_setosa[iris_numeric_columns].describe())
print(iris_versicolor[iris_numeric_columns].describe())
print(iris_virginica[iris_numeric_columns].describe())
print(iris.groupby("species")[iris_numeric_columns].describe())
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	pd.read_csv(...Employee_Salary_Dataset.csv)	Loads employee salary dataset from local device.
2	df.columns = ...	Converts ID, Experience_Years, Age, Gender, Salary into lowercase column names.
3	group_column = "gender"	Selects gender as categorical variable for grouping.
4	numeric_variables = [...]	Selects numeric variables age, experience_years, and salary.
5	df.groupby(group_column)	Groups rows by gender.
6	agg(["mean", "median", "min", "max", "std"])	Calculates central tendency and variability measures.
7	apply(list)	Creates a list of salary values for each gender category.
8	mean(), median(), min(), max(), std()	Calculates basic salary statistics.
9	pd.read_csv(...iris.csv)	Loads Iris dataset from local device.
10	iris.drop("id", axis=1)	Removes ID column if present.
11	iris["species"].unique()	Displays unique Iris species names.
12	select_dtypes(include=[np.number])	Selects only numeric Iris columns.
13	iris[iris["species"] == ...]	Separates data species-wise.
14	describe()	Shows count, mean, std, min, percentiles, and max.
15	groupby("species").describe()	Shows all species-wise statistics together.

Conclusion

The program calculates grouped statistics for salary data and species-wise statistical details for Iris data.

PROGRAM 4	Data Analytics I - Linear Regression
Aim	Build a Linear Regression model to predict house prices using the Boston Housing dataset.
Dataset / File Used	BostonHousing.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\BostonHousing.csv")
df.columns = df.columns.str.strip()

print(df.head())
print(df.info())
print(df.isnull().sum())

for col in df.select_dtypes(include=[np.number]).columns:
    df[col] = df[col].fillna(df[col].median())

if "MEDV" in df.columns:
    target_column = "MEDV"
elif "medv" in df.columns:
    target_column = "medv"
else:
    target_column = df.columns[-1]

X = df.drop(target_column, axis=1)
y = df[target_column]

for col in X.columns:
    if col.lower() == "id":
        X = X.drop(col, axis=1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

comparison = pd.DataFrame({"Actual Price": y_test.values, "Predicted Price": y_pred})
print(comparison.head(10))

mae = mean_absolute_error(y_test, y_pred)
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)

plt.scatter(y_test, y_pred)
plt.xlabel("Actual House Price")
plt.ylabel("Predicted House Price")
plt.title("Actual vs Predicted House Prices")
plt.show()
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	from sklearn.model_selection import train_test_split	Imports function to divide data into training and testing parts.
2	from sklearn.linear_model import LinearRegression	Imports Linear Regression model.
3	from sklearn.metrics import ...	Imports model evaluation functions.
4	pd.read_csv(...BostonHousing.csv)	Loads Boston Housing dataset from local device.
5	df.isnull().sum()	Checks missing values.
6	fillna(median)	Fills numeric missing values.
7	target_column = "MEDV"	Selects house price column as output variable.
8	X = df.drop(target_column, axis=1)	Stores independent/input variables.
9	y = df[target_column]	Stores dependent/output variable.
10	drop id column	Removes ID because it is not useful for prediction.
11	train_test_split(...)	Splits data into training and testing sets.
12	model = LinearRegression()	Creates linear regression model object.
13	model.fit(X_train, y_train)	Trains the model using training data.
14	model.predict(X_test)	Predicts house prices for test data.
15	comparison = pd.DataFrame(...)	Creates table of actual and predicted prices.
16	MAE, MSE, RMSE, R2	Calculates regression model performance measures.

17	plt.scatter(y_test, y_pred)	Plots actual price versus predicted price.

Conclusion

The model learns relation between housing features and price, predicts test data, and evaluates performance using MAE, MSE, RMSE, and R2 score.

PROGRAM 5	Data Analytics II - Logistic Regression
Aim	Implement logistic regression on Social_Network_Ads.csv and compute confusion matrix, TP, FP, TN, FN, accuracy, error rate, precision, and recall.
Dataset / File Used	Social_Network_Ads.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\Social_Network_Ads.csv")
df.columns = df.columns.str.strip()

print(df.head())
print(df.info())
print(df.isnull().sum())

for col in df.select_dtypes(include=[np.number]).columns:
    df[col] = df[col].fillna(df[col].median())
for col in df.select_dtypes(include=["object"]).columns:
    df[col] = df[col].fillna(df[col].mode()[0])

if "Gender" in df.columns:
    encoder = LabelEncoder()
    df["Gender"] = encoder.fit_transform(df["Gender"])

if "User ID" in df.columns:
    df = df.drop("User ID", axis=1)

target_column = "Purchased"
X = df.drop(target_column, axis=1)
y = df[target_column]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
cm = confusion_matrix(y_test, y_pred)
print(cm)

TN = cm[0][0]
FP = cm[0][1]
FN = cm[1][0]
TP = cm[1][1]

accuracy = (TP + TN) / (TP + TN + FP + FN)
error_rate = (FP + FN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)

print("TP:", TP)
print("FP:", FP)
print("TN:", TN)
print("FN:", FN)
print("Accuracy:", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	StandardScaler	Used to scale age and salary values before logistic regression.
2	LabelEncoder	Used to convert Gender into numeric values.
3	LogisticRegression	Used to create a classification model.
4	confusion_matrix	Used to compute TP, FP, TN, and FN.
5	pd.read_csv(...Social_Network_Ads.csv)	Loads Social Network Ads dataset from local device.
6	fillna(median/mode)	Handles missing numeric and categorical values.
7	df["Gender"] = encoder.fit_transform(...)	Converts Male/Female into numbers.
8	df.drop("User ID", axis=1)	Removes User ID because it is not useful for prediction.
9	target_column = "Purchased"	Selects Purchased as output class.
10	X = df.drop(...)	Stores input features.
11	y = df[target_column]	Stores target class.
12	train_test_split(...)	Divides dataset into training and testing data.
13	scaler.fit_transform(X_train)	Fits scaler on training data and transforms it.
14	scaler.transform(X_test)	Transforms test data using training scale.

15	model.fit(...)	Trains logistic regression model.
16	model.predict(...)	Predicts purchased/not purchased for test data.
17	cm = confusion_matrix(...)	Creates confusion matrix in format [[TN, FP], [FN, TP]].
18	TN, FP, FN, TP	Extracts confusion matrix values.
19	accuracy	Calculates correct predictions ratio.
20	error_rate	Calculates wrong predictions ratio.
21	precision	Calculates positive prediction correctness.
22	recall	Calculates actual positive detection rate.

Conclusion

Logistic Regression classifies whether a user purchased or not. The confusion matrix is used to calculate classification performance measures.

PROGRAM 6	Data Analytics III - Naive Bayes Classification
Aim	Implement Naive Bayes classification on iris.csv and compute confusion matrix and classification measures.
Dataset / File Used	iris.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
iris = pd.read_csv(BASE_PATH + r"\iris.csv")
iris.columns = iris.columns.str.strip().str.lower()

if "id" in iris.columns:
    iris = iris.drop("id", axis=1)

print(iris.head())
print(iris.isnull().sum())

for col in iris.select_dtypes(include=[np.number]).columns:
    iris[col] = iris[col].fillna(iris[col].median())

encoder = LabelEncoder()
iris["species"] = encoder.fit_transform(iris["species"])

X = iris.drop("species", axis=1)
y = iris["species"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, average="macro")
recall = recall_score(y_test, y_pred, average="macro")

print("Accuracy:", accuracy)
```

```
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall:", recall)

for i, class_name in enumerate(encoder.classes_):
    TP = cm[i][i]
    FP = cm[:, i].sum() - TP
    FN = cm[i, :].sum() - TP
    TN = cm.sum() - (TP + FP + FN)
    print(class_name, "TP:", TP, "FP:", FP, "TN:", TN, "FN:", FN)
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	GaussianNB	Imports Naive Bayes classifier based on Gaussian distribution.
2	accuracy_score, precision_score, recall_score	Imports functions to evaluate classification model.
3	pd.read_csv(...iris.csv)	Loads Iris dataset from local device.
4	iris.drop("id", axis=1)	Removes ID column if available.
5	fillna(median)	Fills missing numeric values.
6	LabelEncoder()	Creates encoder to convert species names into numbers.
7	encoder.fit_transform(iris["species"])	Converts species names into numeric labels.
8	X = iris.drop("species", axis=1)	Stores input flower measurements.
9	y = iris["species"]	Stores output class labels.
10	train_test_split(...)	Splits dataset into train and test parts.
11	model = GaussianNB()	Creates Naive Bayes model.
12	model.fit(...)	Trains the model.
13	model.predict(...)	Predicts Iris species for test records.
14	confusion_matrix(...)	Creates multiclass confusion matrix.
15	accuracy = accuracy_score(...)	Calculates total correct predictions.
16	error_rate = 1 - accuracy	Calculates total wrong predictions.
17	precision_score(... average="macro")	Calculates average precision across all classes.
18	recall_score(... average="macro")	Calculates average recall across all classes.
19	for i, class_name in enumerate(...)	Calculates TP, FP, TN, FN separately for each Iris class.

Conclusion

Naive Bayes classifies Iris species based on flower measurements and evaluates performance using confusion matrix, accuracy, precision, recall, and class-wise TP, FP, TN, FN.

PROGRAM 7	Text Analytics
Aim	Apply tokenization, POS tagging, stop words removal, stemming, lemmatization, and calculate TF, IDF, and TF-IDF.
Dataset / File Used	Sample text document inside Python

Complete Python Code

```
import nltk
import pandas as pd
import math
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer

nltk.download("punkt")
nltk.download("averaged_perceptron_tagger")
nltk.download("stopwords")
nltk.download("wordnet")
nltk.download("omw-1.4")

document = """
Text analytics is the process of converting unstructured text data into meaningful information.
Students are learning tokenization, stemming, lemmatization, stop word removal and POS tagging.
"""

document_lower = document.lower()
tokens = word_tokenize(document_lower)

words = []
for token in tokens:
    if token.isalpha():
        words.append(token)

pos_tags = nltk.pos_tag(words)
print(pos_tags)

stop_words = set(stopwords.words("english"))
filtered_words = []
for word in words:
    if word not in stop_words:
        filtered_words.append(word)
print(filtered_words)

stemmer = PorterStemmer()
stemmed_words = []
for word in filtered_words:
    stemmed_words.append(stemmer.stem(word))
print(stemmed_words)
```

```
lemmatizer = WordNetLemmatizer()
lemmatized_words = []
for word in filtered_words:
    lemmatized_words.append(lemmatizer.lemmatize(word))
print(lemmatized_words)

documents = [
    "Text analytics converts text data into meaningful information",
    "Students are learning text preprocessing techniques",
    "Tokenization stemming and lemmatization are important text processing steps"
]

processed_documents = []
for doc in documents:
    tokens = word_tokenize(doc.lower())
    clean_words = []
    for token in tokens:
        if token.isalpha() and token not in stop_words:
            clean_words.append(token)
    processed_documents.append(clean_words)

vocabulary = []
for doc in processed_documents:
    for word in doc:
        if word not in vocabulary:
            vocabulary.append(word)

tf_values = []
for doc in processed_documents:
    tf_doc = {}
    for word in vocabulary:
        tf_doc[word] = doc.count(word) / len(doc)
    tf_values.append(tf_doc)

tf_table = pd.DataFrame(tf_values)
print(tf_table)

idf_values = {}
total_documents = len(processed_documents)
for word in vocabulary:
    document_count = 0
    for doc in processed_documents:
        if word in doc:
            document_count += 1
    idf_values[word] = math.log(total_documents / document_count)

idf_table = pd.DataFrame([idf_values])
print(idf_table)
```

```
tfidf_values = []
for tf_doc in tf_values:
    tfidf_doc = {}
    for word in vocabulary:
        tfidf_doc[word] = tf_doc[word] * idf_values[word]
    tfidf_values.append(tfidf_doc)

tfidf_table = pd.DataFrame(tfidf_values)
print(tfidf_table)
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	import nltk	Imports NLTK library for text processing.
2	import math	Imports math library for logarithm used in IDF.
3	word_tokenize	Splits text into words and symbols.
4	stopwords	Provides common stop words like is, the, and.
5	PorterStemmer	Used for stemming words.
6	WordNetLemmatizer	Used for lemmatization.
7	nltk.download(...)	Downloads required NLTK resources.
8	document = ...	Stores sample text document.
9	document.lower()	Converts text to lowercase.
10	word_tokenize(document_lower)	Performs tokenization.
11	token.isalpha()	Keeps only alphabetic tokens and removes punctuation.
12	nltk.pos_tag(words)	Performs POS tagging.
13	stop_words = set(...)	Creates stop words set.
14	if word not in stop_words	Removes stop words from tokens.
15	stemmer.stem(word)	Converts word to stem form.
16	lemmatizer.lemmatize(word)	Converts word to meaningful root form.
17	documents = [...]	Creates multiple documents for TF-IDF.
18	processed_documents.append(...)	Stores cleaned words of each document.
19	vocabulary = []	Stores all unique words.
20	doc.count(word) / len(doc)	Calculates term frequency.
21	math.log(total_documents / document_count)	Calculates inverse document frequency.
22	tf_doc[word] * idf_values[word]	Calculates TF-IDF value.

Conclusion

The program preprocesses text and creates TF-IDF representation to measure word importance in documents.

PROGRAM 8	Data Visualization I - Titanic Dataset
Aim	Use Seaborn to find patterns in Titanic data and plot fare distribution histogram.
Dataset / File Used	Titanic1.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\Titanic1.csv")
df.columns = df.columns.str.strip().str.lower()

print(df.head())
print(df.info())
print(df.isnull().sum())

df["age"] = df["age"].fillna(df["age"].median())
df["fare"] = df["fare"].fillna(df["fare"].median())
df["embarked"] = df["embarked"].fillna(df["embarked"].mode()[0])

sns.countplot(x="survived", data=df)
plt.title("Survival Count")
plt.show()

sns.countplot(x="sex", hue="survived", data=df)
plt.title("Survival Count Based on Gender")
plt.show()

sns.countplot(x="class", hue="survived", data=df)
plt.title("Survival Count Based on Passenger Class")
plt.show()

sns.histplot(df["age"], bins=30, kde=True)
plt.title("Age Distribution of Passengers")
plt.show()

sns.boxplot(x="class", y="fare", data=df)
plt.title("Fare Distribution Based on Passenger Class")
plt.show()

plt.hist(df["fare"], bins=30)
plt.title("Distribution of Ticket Fare")
plt.xlabel("Fare")
plt.ylabel("Number of Passengers")
plt.show()
```

```
sns.histplot(df["fare"], bins=30, kde=True)
plt.title("Distribution of Ticket Fare")
plt.show()
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	import seaborn as sns	Imports Seaborn for statistical graphs.
2	pd.read_csv(...Titanic1.csv)	Loads Titanic dataset from local device.
3	df.columns = ...	Cleans column names.
4	df.isnull().sum()	Checks missing values.
5	df["age"].fillna(median)	Fills missing age values using median.
6	df["fare"].fillna(median)	Fills missing fare values using median.
7	df["embarked"].fillna(mode)	Fills missing embarked value using mode.
8	sns.countplot(x="survived")	Shows count of survived and not survived passengers.
9	sns.countplot(x="sex", hue="survived")	Shows survival pattern based on gender.
10	sns.countplot(x="class", hue="survived")	Shows survival pattern based on passenger class. Use class, not pclass, for your CSV.
11	sns.histplot(df["age"], kde=True)	Shows distribution of passenger ages.
12	sns.boxplot(x="class", y="fare")	Shows fare distribution class-wise.
13	plt.hist(df["fare"], bins=30)	Plots histogram of ticket fare.
14	sns.histplot(df["fare"], bins=30, kde=True)	Plots Seaborn histogram with smooth KDE curve.

Conclusion

The Titanic dataset is visualized using countplots, histograms, and boxplots to find survival and fare patterns.

PROGRAM 9	Data Visualization II - Titanic Boxplot
Aim	Plot a boxplot for distribution of age with respect to gender and survival status.
Dataset / File Used	Titanic1.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\Titanic1.csv")
df.columns = df.columns.str.strip().str.lower()

df["age"] = df["age"].fillna(df["age"].median())

print(df["sex"].unique())
print(df["survived"].unique())

plt.figure(figsize=(8, 6))
sns.boxplot(x="sex", y="age", hue="survived", data=df)
plt.title("Age Distribution by Gender and Survival")
plt.xlabel("Gender")
plt.ylabel("Age")
plt.legend(title="Survived", labels=["Not Survived", "Survived"])
plt.show()
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	pd.read_csv(...Titanic1.csv)	Loads Titanic dataset from local device.
2	df.columns = ...	Cleans column names.
3	df["age"].fillna(df["age"].median())	Fills missing age values using median.
4	df["sex"].unique()	Displays gender categories.
5	df["survived"].unique()	Displays survival values 0 and 1.
6	plt.figure(figsize=(8, 6))	Sets graph size.
7	sns.boxplot(x="sex", y="age", hue="survived", data=df)	Creates boxplot of age grouped by sex and separated by survival.
8	plt.title(...)	Adds graph title.
9	plt.xlabel("Gender")	Adds x-axis label.
10	plt.ylabel("Age")	Adds y-axis label.

11	plt.legend(...)	Explains 0 as not survived and 1 as survived.
12	plt.show()	Displays the graph.

Observations / Inference

- The boxplot compares age distribution of male and female passengers.
- The hue value separates survived and not-survived passengers.
- Female passengers generally show better survival pattern than male passengers.
- The middle line in each box shows median age.
- Outlier points represent passengers with extreme ages.

Conclusion

The boxplot helps compare age distribution by gender and survival status.

PROGRAM 10	Data Visualization III - Iris Feature Distributions
Aim	List feature types, plot histograms and boxplots for Iris features, compare distributions, and identify outliers.
Dataset / File Used	iris.csv from local device

Complete Python Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

BASE_PATH = r"C:\Users\Avadhoot\Downloads\DSBDAL TUTORIALS"
df = pd.read_csv(BASE_PATH + r"\iris.csv")
df.columns = df.columns.str.strip().str.lower()

print(df.head())
print(df.info())
print(df.isnull().sum())

numeric_features = df.select_dtypes(include=[np.number]).columns.tolist()
categorical_features = df.select_dtypes(include=["object"]).columns.tolist()

if "id" in numeric_features:
    numeric_features.remove("id")

feature_types = []
for col in df.columns:
    if col in numeric_features:
        feature_types.append([col, "Numeric"])
    else:
        feature_types.append([col, "Nominal / Categorical"])

feature_table = pd.DataFrame(feature_types, columns=["Feature Name", "Feature Type"])
print(feature_table)

for col in numeric_features:
    sns.histplot(df[col], bins=20, kde=True)
    plt.title("Distribution of " + col)
    plt.show()

for col in numeric_features:
    sns.boxplot(y=df[col])
    plt.title("Boxplot of " + col)
    plt.show()

sns.boxplot(data=df[numeric_features])
plt.title("Boxplot of All Numeric Features")
plt.show()
```

```
def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR
    return data[(data[column] < lower_limit) | (data[column] > upper_limit)]

for col in numeric_features:
    outliers = detect_outliers_iqr(df, col)
    print("Feature:", col)
    print("Number of outliers:", len(outliers))
    print(outliers[[col]])

print(df[numeric_features].describe())
```

Line-by-Line Explanation

Step	Code Line / Block	Explanation
1	pd.read_csv(...iris.csv)	Loads Iris dataset from local device.
2	df.columns = ...	Cleans column names.
3	df.info()	Shows feature names and data types.
4	numeric_features = ...	Selects numeric features.
5	categorical_features = ...	Selects nominal/categorical features.
6	numeric_features.remove("id")	Removes ID column if present.
7	feature_types = []	Creates list for feature name and feature type.
8	for col in df.columns	Loops through all columns.
9	feature_table = pd.DataFrame(...)	Creates feature type table.
10	sns.histplot(df[col], bins=20, kde=True)	Creates histogram for each numeric feature.
11	sns.boxplot(y=df[col])	Creates separate boxplot for each numeric feature.
12	sns.boxplot(data=df[numeric_features])	Creates combined boxplot for all numeric features.
13	quantile(0.25), quantile(0.75)	Calculates Q1 and Q3.
14	IQR = Q3 - Q1	Calculates interquartile range.
15	lower_limit and upper_limit	Defines outlier boundaries.
16	return data[(...)]	Returns rows having outliers.
17	describe()	Shows statistical summary for numeric features.

Observations / Inference

- Iris dataset contains numeric features like sepal length, sepal width, petal length, and petal width.

- Species is a nominal/categorical feature.
- Histograms show feature distributions.
- Boxplots show median, spread, and outliers.
- Sepal width usually shows more outliers compared with other features.
- Petal length and petal width are very useful for separating Iris species.

Conclusion

The Iris features are visualized using histograms and boxplots, and outliers are identified using the IQR method.

Quick Viva Revision Points

Concept	Simple Viva Answer
Data Wrangling	Cleaning, formatting, transforming, and preparing raw data for analysis.
Missing Values	Median is used for numeric columns; mode is used for categorical columns.
Outliers	IQR detects values outside Q1 - 1.5*IQR and Q3 + 1.5*IQR.
Normalization	Changes values to a common scale, commonly 0 to 1.
Encoding	Label encoding converts categories into numeric codes; one-hot encoding creates binary columns.
Linear Regression	Predicts continuous numeric values such as house price.
Logistic Regression	Performs classification, such as purchased or not purchased.
Naive Bayes	A probabilistic classification algorithm based on Bayes theorem.
TF-IDF	Measures word importance in documents using term frequency and inverse document frequency.
Histogram	Shows distribution/frequency of values.
Boxplot	Shows median, quartiles, spread, and outliers.